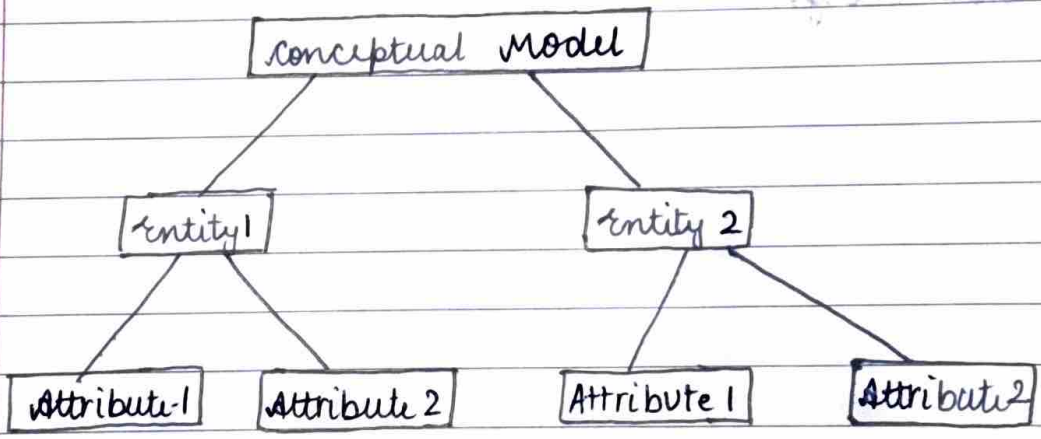
Unit → 3

→ Database design stratergy

1. Top down database design
2. Bottom up database design
3. E-R model database design
4. Relational database design
5. Object Oriented database design
6. Centralized / Distributed database design
7. Normalization database design
8. Denormalization database design
9. Dataware house database design
10. Inside out database design

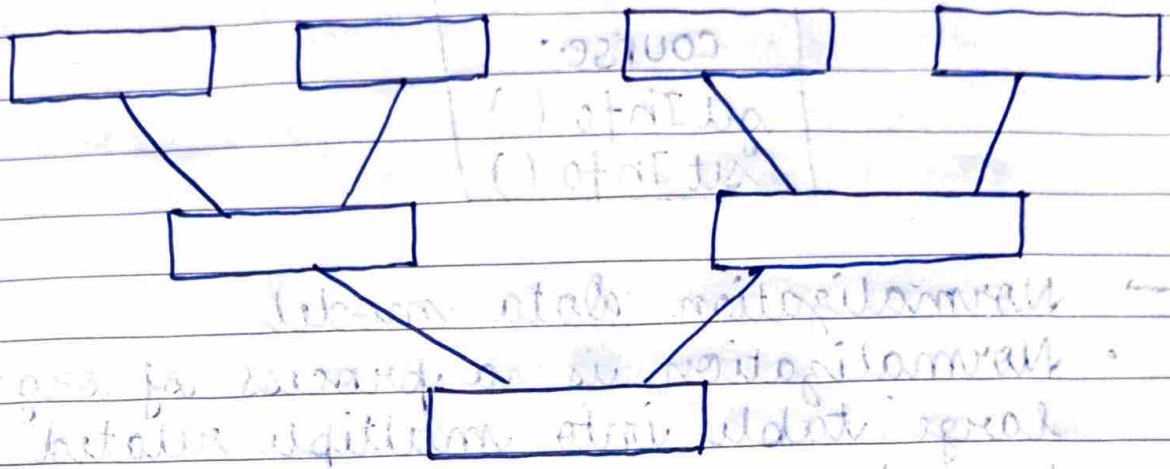
- Database design stratergy is a process of deciding how the data should be organised, what exactly data should be stored and how different data items are related.

→ Top down database design



- Top down approach starts with complete ~~overall~~ view of the system and gradually break down to smaller components

→ Bottom up Database Design
starts with individual attributes and gradually combine them into entities and tables



→ Relational Data model
- Data is represented using tables
- eg → MY SQL
- Tables are also called relation that stores data into tuples and attributes
- A tuple in a table / relation represents the relationship among the set of values and each attribute must have atomic value
  - only 1 value in a tuple of a table
  - eg → ID (primary key)

→ Object oriented data model
- Data and information is represented in form of object
- These model is basically used for multi media application where information is stored in form of image, audio & video
- It's the extension of ER model & used to represent real world entities

eg :-

STUDENT
Attributes ID Name COURSE
get Info () set Info ()

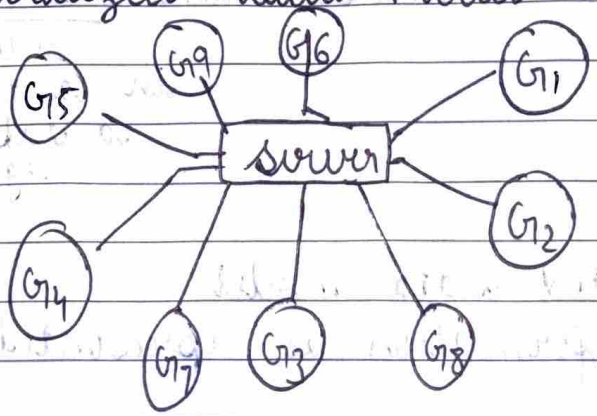
→ Normalization data model

- Normalization is a process of organizing large table into multiple related tables to reduce the data redundancy

→ Denormalization data Model

- Denormalization is a process of combining table of some repetitive data to improve query performance & provide faster access of data

→ Centralized Data Model



- In a centralized database, data is stored & managed at one central location
- Data is stored in server and is available to connected computers

→ Distributed data model
- Database in which several datas are distributed over several servers but it acts like a single database to the user
- It gives high reliability, better local speed and no single point of failure

→ Data warehouse Data model
- A large central repository that connects and collects historical data from multiple sources for business reporting and analysis
- It helps in fast analytics, trend forecasting & executive decision making

→ Inside out Design
- A database design approach that starts with specific, low level details and works outward to build bigger system
- It's very practical and grounded in real operational details though combining the pieces later that may cause minor conflicts.

→ Functional Dependency

- Describes a relationship between a set of attributes (columns of a table) in a relation that helps to remove redundancy. It's represented as $X \rightarrow Y$ → dependent, determinant

1. Trivial Functional Dependency

$A \rightarrow B$ & $B \subseteq A$ [A determines smth it already contains]

eg → (Student ID, student name) → (student name)

2. Non trivial Functional Dependency

$A \rightarrow B$ & $B \nsubseteq A$

eg → (ID, name) → (date of birth)

- If the right hand side isn't a subset of left hand side

3. Partial Functional Dependency

$A, B \rightarrow C$ A & B → candidate keys
$C \subseteq B$ C → non key

eg → Any one of A & B needed to find C
(student id, course id) → (name)

also

student id → name ✓
but course id → name ✗
} partially dependent

4. Fully Functional Dependency

$A, B \rightarrow C$ A & B → candidate keys

Both A & B needed to find C. C → non key

eg → (student id, course id) → (marks)

5. Transitive Functional dependency
If $A \rightarrow B$ and $B \rightarrow C$
then $A \rightarrow C$
eg :-
If student id → dept id
dept id → dept name
than student id → dept name

- occurs when an attribute depends on another non-key attribute through intermediate attribute

→ Functional Dependency

ID	NAME	ADDRESS	DOB
101	Ronit	C-56	17 Sep
102	Sonal	A-23	2 Feb
103	Amit	B-92	3 Jul

1. α → same ⇒ β should be same for $\alpha \rightarrow \beta$ be valid FD
2. α → different ⇒ β → same / different

eg :-

A	B	C	D	E
a	2	3	4	5
2	a	3	4	5
a	2	3	6	5

check FD (α) → (β) ← don't care

1. $A \rightarrow BC$ ✓valid $A = a$, $B = 2$ & $C = 3$ always && $a = 2$ B & C → X
2. $DE \rightarrow C$ → valid → $C = 3$ $D = 6$ & $E = 5$
3. $C \rightarrow DE$ → invalid
4. $BC \rightarrow A$ → valid

→ closure (X^+)

- It's mainly used with functional dependencies to find all the attributes that can be determined from a given set of attributes
- It's also used to find candidate keys

Q R (A, B, C, D, E) with FD A → B, B → C, CD → E

↳ a table with attributes A, B, C, D, E

A^+ → A, B, C

↳ A ko use krke A aaya, A → B se B aaya aur B → C se C but D, E nhi aya to usa dhundo jisme sare aajaye aur wohi wala closure candidate key hoyega

AD^+ → A, D, B, C, E

↳ candidate key

Q R (A, B, C, D, E) with FD A → B, B → C, CD → E, E → A

A^+ → A, B, C

AD^+ → A, D, B, C, E

↳ candidate key

Q R (A, B, C, D, E, , G) A → B, BC → DE, AEG → G

$(AC)^+$ → B, A, C, DE

$(ACG)^+$ → ACG BDE

→ Minimal set and canonical form

- Minimal set FD ka wo set hai jo sabse simplest aur least h jisse bina redundancy ke closure aajaye

→ steps :-

- Break RHS into single attributes

eg → D → ABC ⇒ D → A, D → B, D → C

- Ek-Eki krke FD ko hatakr dekho aur bakio se closure nikalo agar same aya to usi discard kardo
- Once you discard a FD, don't include it any further

eg → R(A, B C D) A → B C → B D → ABC AC → D

FD's

✓ A → B $(A^+) = \{A\}$ whereas yhi $A^+ = A, B$ tha

✓ C → B $(C^+) = \{C\}$ yhi $C^+ = C, B$

✓ D → A $(D^+) = \{D, B, C\}$ yhi $D^+ = A, B, C, D$

✗ D → B $(D^+) = \{A, B, C, D\}$ → same

✓ D → C $(D^+) = \{A, B, D\}$ → yhi $D^+ = ABCD$

AC → D

Is case me check karo ki kisi ek ko hatane se kam chal rha hai kya

$A^+ = A, B$ → can't give D

$C^+ = C, B$ → can't give D

⇒ Both required

AC → D ✓

Minimal cover :-

A → B

C → B

D → A

D → C → D → AC

AC → D

Q R (WXYZ)

$X \to W$, $WZ \to XY$, $Y \to WXZ$

✓ $X \to W$ $X^+ = X$ else X, W tha

$WZ \to X$ $W^+ = W$ $Z^+ = Z$ need both WZ

✓ $WZ \to Y$ $(WZ^+) = X, W, Z,$ earlier XWZY tha

$Y \to W$ $Y^+ = Y; X, Z, W$ same

$Y \to X$ $Y^+ = Y, W, Z, X$ same

✓ $Y \to Z$ $Y^+ = X, W, Y$ else YXZW tha

Minimal cover :-

$X \to W$, $WZ \to Y$, $Y \to Z$

→ A canonical cover is also called minimal cover is the simplest equivalence set of functional dependencies

→ It represents the same dependency as the original set but removes redundant attributes and redundant FD's.

→ equivalence set of FD

- It's the two distinct sets of FD, F and G that express the exact same logical constraints over a relational database schema
- Sets of 2 functional dependencies F & G are equivalent ($F \equiv G$) if and only if every functional dependency of F can be inferred from G and every FD in G can be inferred from F

i.e. if $F \subseteq G^+$ & & $G \subseteq F^+$ $\Rightarrow$ $G \equiv F$

Q R(VWXYZ)

F

$V \rightarrow W$

$W \rightarrow X$

$Y \rightarrow VX$

$Y \rightarrow Z$

$V^+ \rightarrow VWX$

$W^+ \rightarrow W$

$Y^+ \rightarrow VXWYZ$

G

$V \rightarrow W$

$V \rightarrow X$

$Y \rightarrow V$

$Y \rightarrow Z$

$V^+ \rightarrow VWX$

$Y^+ \rightarrow VZWXY$

Hence we can find all FD of F using G & vice versa

$\Rightarrow F \equiv G$

Q R(ABCDE)

F

$B \rightarrow CDA$

$AD \rightarrow E$

$B^+ \rightarrow BCDAE$

$AD^+ \rightarrow EAD$

$D^+ \rightarrow DE$

G

$B \rightarrow CDE$

$D \rightarrow BC$

$AD \rightarrow E$

$B^+ \rightarrow BCDE$

$D^+ \rightarrow BCD$

$AD^+ \rightarrow ADE$

F me $B \rightarrow A$ tha jo G se nhi aarha also

G me $D \rightarrow B$, $D \rightarrow C$ tha jo F se nhi aarha

$\Rightarrow F \not\equiv G$

not equivalent

→ Normalization

STUDENT ID	NAME	BRANCH ID	BRANCH NAME	CHAIR PERSON
101	Sonia	C1	CSE	Dr Suman
102	Anaisha	C1	CSE	Dr Suman
103	Mahi	C3	AI	Dr Suman
104	Deepak	C1	CSE	Dr Suman

Normalization is the process of reorganization or decomposing of data / table so that every table must contain single idea and if two informations aren't directly related to each other then they must be in different tables



→ Problems (Anomalies) without normalization

. Anomaly is a problem that occurs in a database, when data isn't properly organized

1. Insert Anomaly

. When you can't insert new information without having such unrelated information

eg → course can only be added in student table when there is atleast one student of that branch ∵ ID → primary key but in real circumstances ideally branch can be added with / without any student belonging to it.

2. Update Anomaly
- When same information is repeated multiple times in multiple tuples then we have to update it everywhere

3. Delete Anomaly
- When we delete one piece of information, we accidently remove the other important piece of information
eg → If we delete data of student that belongs to a branch having only one student (A1 in this case) we end up loosing the data of branch also.

→ Normal Forms in DBMS

1. 1NF
- Must have a primary key
- No two columns / attributes have same name
- Every cell has single atomic value
  ↳ 1 entry / cell

2. 2NF
- It should be in 1NF
- It shouldn't contain partial dependency
  ↓
  when a non prime attribute can be derived from any other key except candidate key

eg :-
R(ABCD)

candidate key ← AB → C → non prime attributes
B → D
↳ Prime attribute

As B → D ⇒ Not 2NF

3. 3NF

- Must be in 2NF
- Should not contain transitive dependency

R(ABCD)

$A \rightarrow B \rightarrow$ non prime

$B \rightarrow C$ [driving a non prime attribute using a non prime attribute]

$\Rightarrow$ 3NF X

→ Decomposition (lossy / lossless)

- A disadvantage of normalization is that sometimes, no of tuples increase in database because of which access time ↑ & system becomes slow while child tables are combined to get parent table
- If we get the original parent table then it means decomposition is lossless
- But if we don't get the original parent table on combining child tables then it's lossy decomposition
- Every decomposition during normalization must be lossless and it's mandatory policy that must be satisfied

→ How to identify lossy / lossless decomp.

- Try to find the common attribute, if there is no common attribute then it's lossy decomposition
- Common attribute must have unique value If yes → lossless otherwise lossy.

4. BCNF (Boyce Codd Normal Form)

If $X \rightarrow Y$

then X must be a super key

eg → R(ABCDEF).

$A \rightarrow BC$, $D \rightarrow E$, $AD \rightarrow F$

Finding candidate key :-

$(AD)^+ = \{A\ D\ B\ C\ E\ F\} = R$

⇒ AD is primary candidate key

checking for BCNF :-

$(A)^+ = \{A, B, C\}$

⇒ A isn't super key (cant find def)

$(D)^+ = \{D, E\}$

⇒ D isn't super key

$(AD)^+ = \{A, B, C, D, E, F\}$

⇒ AD is super key

Since $A \rightarrow BC$ & $D \rightarrow E$ but A & D are not super keys

⇒ R isn't in BCNF

- A relation is in BCNF for every non trivial FD $X \rightarrow Y$, X must be a super key

If one attribute / multiple attributes determined on other attributes then it comes under BCNF

5. 4NF

A relation is in 4NF if :-

1. It's in BCNF
2. It must not contain, non-trivial multi valued FD

STUDENT	SKILL	HOBBY
Rahul	Python	cricket
Rahul	Python	chess
Rahul	Java	cricket
Rahul	Java	chess

→ NOT in 4NF

1. Skill aur hobby ka apas me koi lena dena nhi hai lekin fir bi ek table me store krne ki wjh se redundancy aagyi
2. New hobby add krne ke lie 2 new rows chahiye (update anomaly)

↳ This (1, 2) is called multivalued FD ($X \rightarrow\rightarrow Y$)

6. 5NF

- A relation is in 5NF if :-

1. It's already in 4NF
2. Has no non trivial joint dependency that aren't implied by candidate key
- It can't be further losslessly decomposed.

→ BCNF Decomposition Algorithm

- Find the candidate key
- If any FD $X \to Y$ violates BCNF (i.e. X isn't super key), then break R into two parts

$$R \nearrow R_1 = X^+$$
$$R \searrow R_2 = (R - X^+) \cup X$$

- Repeat until all decomposed forms (R_1 & R_2) become BCNF

eg → R(ABCD)

$A \to B$ $B \to C$

Now $A^+ = \{A, B, C\}$

↳ not a super key

⇒ $R_1 = A^+ = (A, B, C)$

& $R_2 = (R - A^+) \cup A$

$= (\{ABCD\} - \{ABC\}) \cup A$

$= \{D\} \cup \{A\}$

$= \{A, D\}$

⇒ $R_2 = (AD)$

Checking: R_1

$A \to B$

$A^+ = \{A, B, C\}$

↳ super key ✓

$B \to C$

$B^+ = \{B, C\}$

↳ Not super key

↳ violates BCNF

Checking R_2

No non trivial FD exist for R_2

⇒ R_2 is in BCNF

(todne ke bad wohi FD check hogi jinme sirf R_2 ke element hai) R_2 {A, D) hai aur $A \to B$ ek FD thi. pr R_2 me B ni hai to usse frk ni pdta

$R_1 \nearrow R_3$
$R_1 \searrow R_4$

$R_3 = B^+ = \{B, C\} = (BC)$

$R_4 = (R_1 - B^+) \cup B = A \cup B = (AB)$

$\Rightarrow R$ decomposed to (AB) (BC) (AD)

Q $R(A, B, C, D)$

$AB \rightarrow C \quad C \rightarrow D$

$AB^+ = \{A, B, C, D\}$

↳ super key

$C^+ = \{C, D\}$

↳ violates BCNF

$R_1 = C^+ = \{C, D\}$

$R_2 = (R - C^+) \cup C = (AB) \cup C = \{A, B, C\}$

$R_1 \rightarrow$ in BCNF ✓

R_2 in BCNF ✓

Q $R(A, B, C, D, E)$

$A \rightarrow B \quad C \rightarrow D$

$A^+ = \{A, B\} \qquad C^+ = \{C, D\}$

↳ violates BCNF

$R_1 = A^+ = (A, B) \rightarrow$ BCNF ✓

$R_2 = (R - A^+) \cup A = (A, C, D, E)$

In R_2

$C \rightarrow D$

but $C^+ = (C, D) \rightarrow$ violates BCNF

$R_3 = C^+ = (C, D)$

$R_4 = (R_2 - C^+) \cup C = (A, C, E)$

Final $\rightarrow$ (A, B), (C, D), (A, C, E)

→ 3NF Normalization Algorithm

1. Find canonical cover
2. canonical cover ki har FD $X \to A$ ke lie (X, A) relation bnao, same LHS wali FD's ko merge kardo
3. Agr kisi bhi relation me R ki candidate key nhi hai to alg relation add kro jo sirf ek candidate key ka ho
4. Jo relation kisi dusre ka subset hai use hata do.

eg → R(A B C D)

$A \to BC$ $B \to C$ $AB \to D$

✓ $A \to B$ $A^+ = \{A, C\}$ but phle $\{A, B, C, D\}$ tha

$A \to C$ $A^+ = \{A, B, C, D\}$ → redundant

✓ $B \to C$ $B^+ = \{B\}$ phle $\{B, C\}$ tha

$AB \to D$

$A^+ = \{A, B, C, D\}$ → can give D alone ; drop B

$B^+ = \{B, C\}$ ⇒ $A \to D$ ✓

canonical cover :-

$A \to B$ $B \to C$ $A \to D$

relations :-

(A, B) (B, C) (A, D)

Merge

$R_1(A, B, D)$ & $(B, C) R_2$

Also candidate key of R is A

since $A^+ = \{A, B, C, D\}$

& A is in R_1

⇒ decomposed form ⇒ $R_1 = (ABD)$

$R_2 = (BC)$

Q R (A, B, C, D, E)

$A \to B$ $BC \to D$ $D \to E$

canonical cover

✓ $A \to B$ $A^+ = \{A\}$ but yhi $\{A, B\}$

✓ $BC \to D$ $B^+ = \{B\}$ $C^+ = \{C\}$ → essential (can't find D)

✓ $D \to E$ $D^+ = \{D\}$

yhi $\{D, E\}$ tha

canonical cover →

$A \to B$ $BC \to D$ $D \to E$

Relations

R_1 (A, B) R_2 (B, C, D) R_3 (D, E)

candidate key of R is

$A^+ = A, B$

$AC^+ = A\ C\ B\ D\ E$

⇒ AC → candidate key of R

But kisi bhi R me (A, C) nhi hai to add $R_4 = (A, C)$

Finally :-

(A, B) = R_1 (B, C, D) = R_2 (D, E) = R_3

and (A, C) = R_4

Q R(A B C D) $A \to B$ $A \to C$ $C \to D$

candidate key = A as $A^+ = \{A, B, C, D\}$

closure :-

$A \to B$ $A^+ = \{A, C, D\}$ ✓

$A \to C$ $A^+ = \{A, B, D\}$ ✓

$C \to D$ $C^+ = \{C\}$ ✓

Closure :-

$A \rightarrow B$, $A \rightarrow C$, $C \rightarrow D$

Relations :-

(A, B) (A, C) (C, D)

↓ Merge

= (A, B, C) and (C, D)

↓

candidate key involved ✓